SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1003	**Course Title:**	Software Engineering
CO Assessed:	CO3	**Assessment:**	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	**Total Marks:**	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	K Y V Satyanarayana reddy	**Reg. No.:**	192472157
Date:	22.08.2026	**Duration:**	60 minutes

Code of Conduct

I certify that K Y V Satyanarayana Reddy (192472157) this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K Y V Satyanarayana reddy

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. **Problem Analysis**
   - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

   Problem Statement

   The Travel Expense Tracking and Budget Management Application is designed to help travelers manage their travel expenses and budgets efficiently. Traditional methods such as notebooks and spreadsheets can make expense recording, calculation, and budget monitoring difficult. The proposed system provides a centralized application for recording expenses, managing trip budgets, monitoring spending, and generating reports.

   Project Objectives

   ✓ To simplify travel expense tracking and budget management.

   ✓ To allow users to create trips and set planned budgets.

   ✓ To record and categorize travel expenses accurately.

- ✓ To automatically calculate total expenses and remaining budget.
- ✓ To provide budget alerts when spending approaches or exceeds the limit.
- ✓ To generate expense reports and summaries for better financial analysis.

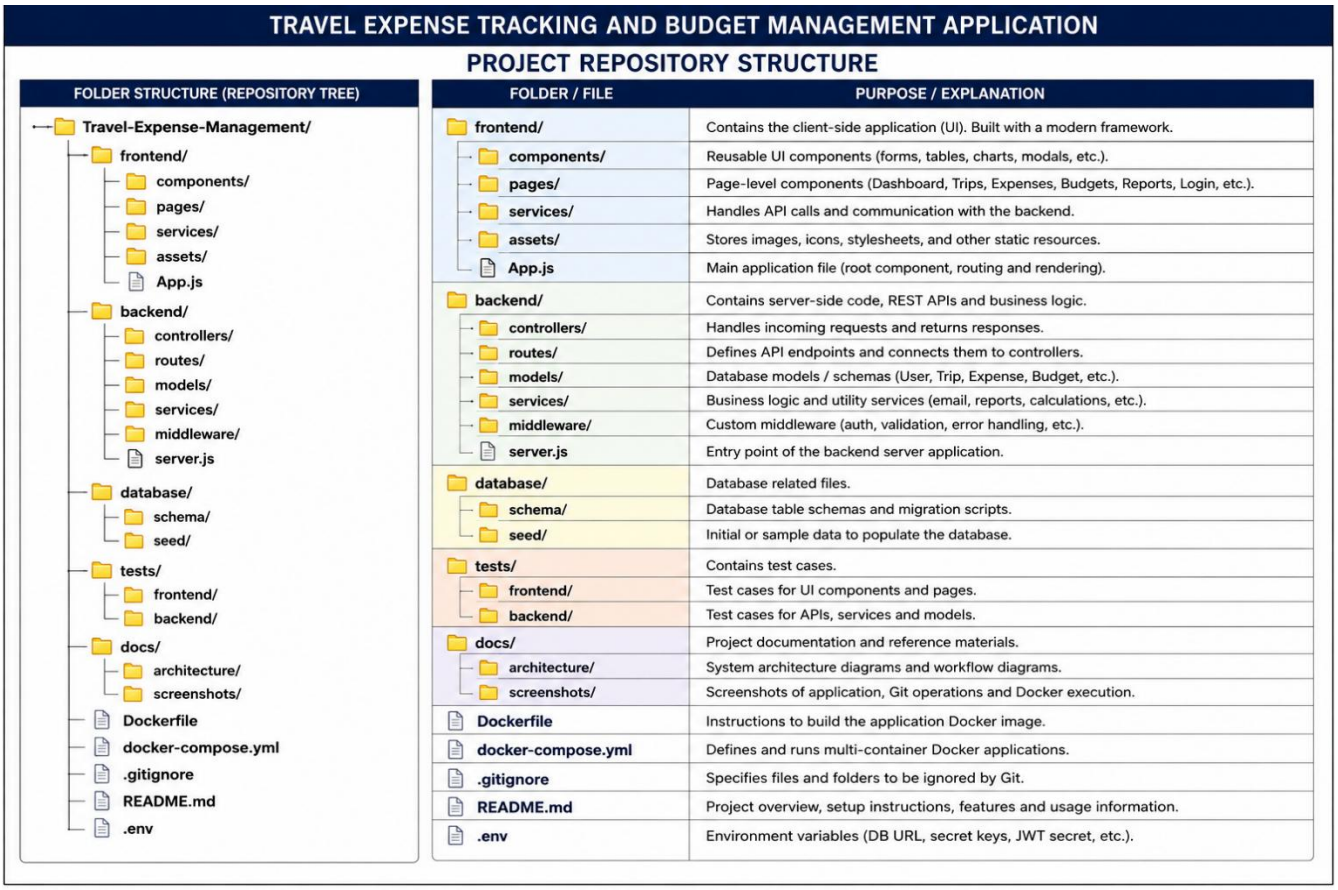
Functional Requirements

- User registration and secure login.
- Trip creation and management.
- Budget creation and updating.
- Adding, editing, deleting, and viewing expenses.
- Categorizing expenses such as food, transportation, accommodation, and shopping.
- Automatic calculation of total expenses and remaining budget.
- Budget monitoring and alerts.
- Generation of expense reports and summaries.
- Secure storage of travel and expense records.

Expected Outcomes

- Reduced manual effort and calculation errors.
- Accurate and organized travel expense records.
- Better real-time control over travel budgets.
- Easy generation of expense reports.
- Improved financial planning and spending decisions.
- Secure and convenient management of travel expenses.

2. **Project Structure Design**
    - Design and explain the folder structure of your project repository.



- Justify the purpose of each major folder and file included in the repository.

- **frontend/** – Contains the client-side interface of the application, including pages, components, services, and visual resources.
- **components/** – Stores reusable UI elements such as forms, tables, charts, and dialogs.
- **pages/** – Contains application pages such as Dashboard, Trips, Expenses, Budgets, Reports, and Login.
- **services/** – Handles API requests and communication between the frontend and backend.
- **assets/** – Stores images, icons, stylesheets, and other static resources.
- **App.js** – Acts as the main frontend application file for routing and rendering.
- **backend/** – Contains server-side code, REST APIs, and business logic.
- **controllers/** – Processes client requests and sends appropriate responses.
- **routes/** – Defines API endpoints and connects them to controllers.
- **models/** – Defines database models for users, trips, expenses, and budgets.
- **services/** – Handles business logic such as calculations, reports, and notifications.
- **middleware/** – Handles authentication, validation, and error processing.
- **server.js** – Serves as the main entry point of the backend application.
- **database/** – Contains database-related files.
- **schema/** – Defines database tables and migration structures.

- **seed/** – Contains initial or sample data for the database.
- **tests/** – Contains test cases for validating application functionality.
- **docs/** – Stores project documentation, architecture diagrams, and screenshots.
- **Dockerfile** – Contains instructions for building the application's Docker image.
- **docker-compose.yml** – Defines and manages multiple Docker services and their communication.
- **.gitignore** – Specifies files and folders that should not be tracked by Git.
- **README.md** – Provides project information, setup instructions, features, and usage details.
- **.env** – Stores environment-specific configuration and sensitive variables such as database URLs and secret keys.

3. **Git Repository Initialization**
    - Create a Git repository for the project.

The project repository can be created using the following steps:

1. Create the project folder and open it in the terminal.
2. Initialize Git using:

    ```
    git init
    ```

3. Add all project files to the staging area:

    ```
    git add .
    ```

4. Create the initial commit:

    ```
    git commit -m "Initial project setup"
    ```

5. Verify that the repository has been initialized:

    ```
    git status
    ```

6. If a remote repository such as GitHub is used, connect it using:

    ```
    git remote add origin <repository-url>
    ```

7. Push the project to the remote repository:

    ```
    git branch -M main
    git push -u origin main
    ```

Purpose: Git provides version control for the Travel Expense Tracking and Budget Management Application, allowing changes to be tracked, maintained, and shared throughout development. The assignment specifically requires a Git repository with a complete commit history.

- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

  Repository Initialization Steps

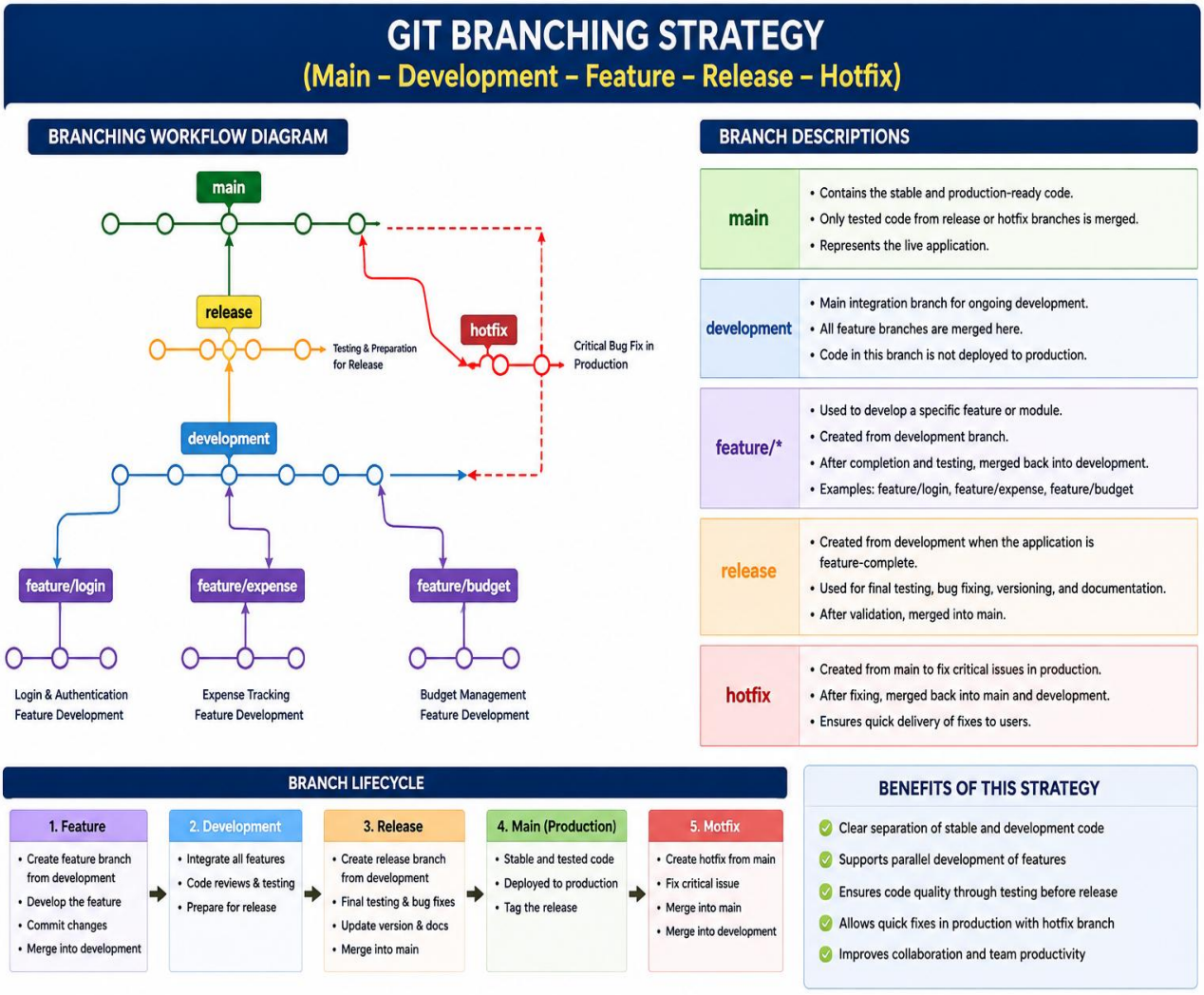
  1. **Create the Project Folder:** Create the Travel-Expense-Management project folder and open it in the terminal.
  2. **Initialize Git:** Run git init to create a new Git repository.
  3. **Check Repository Status:** Run git status to view untracked and modified files.
  4. **Stage Files:** Run git add . to add the project files to the staging area.
  5. **Create Initial Commit:** Run git commit -m "Initial project setup" to save the first version.
  6. **Connect Remote Repository:** Use git remote add origin <repository-url> if the project is hosted on GitHub.
  7. **Synchronize Repository:** Use git push -u origin main to upload the committed project to the remote repository.

  Essential Git Files and Their Purpose

  - **.git/** – The hidden Git directory created by git init; it stores repository history, configuration, branches, and other Git metadata.
  - **.gitignore** – Specifies files and folders that Git should not track, such as .env, temporary files, and dependencies.
  - **README.md** – Provides information about the project, installation steps, features, and usage.
  - **.env** – Stores environment-specific configuration such as database URLs and secret keys; it should normally be excluded from Git tracking.

  These initialization and version-control steps directly support the assignment requirement to create and explain a Git repository and its essential files.

4. **Git Branching Strategy**
   - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).



- Justify why the selected branching model is appropriate for the assigned project.

- **Clear Separation:** The main branch contains stable code, while development is used for ongoing development.
- **Parallel Development:** Separate feature branches allow developers to work on login, expense tracking, and budget management without affecting the main code.
- **Controlled Releases:** The release branch provides a separate stage for final testing and bug fixing before deployment.
- **Quick Bug Fixes:** The hotfix branch allows critical production issues to be corrected quickly and merged back into both main and development.
- **Better Collaboration:** The structured branching model allows multiple developers to work independently and merge their completed work systematically.
- **Improved Code Quality:** Code can be reviewed and tested in development and release branches before reaching the stable main branch.

5. **Version Control Workflow**

- Demonstrate the Git workflow by performing meaningful commits, branch creation,

merging, conflict resolution (if applicable), and repository synchronization.

1. Create and Commit Initial Project

git init

git add .

git commit -m "Initial project setup"

This creates the repository and stores the initial project structure.

2. Create a Development Branch

git checkout -b development

The development branch is used for ongoing project development without directly modifying main.

3. Create a Feature Branch

git checkout -b feature/expense-tracking

This branch is used to develop the expense-tracking functionality independently.

4. Make Changes and Commit

git add .

git commit -m "Add expense tracking feature"

The commit records the completed expense-tracking changes with a meaningful message.

5. Merge Feature into Development

git checkout development

git merge feature/expense-tracking

The completed feature is integrated into the development branch.

6. Resolve a Conflict (If Applicable)

If Git reports a conflict:

git status

Open the conflicting file, manually select the correct changes, and then run:

git add .

git commit -m "Resolve merge conflict"

7. Create a Release Branch

git checkout -b release/v1.0

The release branch is used for final testing and preparation.

8. Merge Release into Main

git checkout main

git merge release/v1.0

9. Synchronize with Remote Repository

git push origin main

git push origin development

10. Verify the Workflow

git log --oneline --graph --all

git branch

git status

Workflow:

Feature → Development → Release → Main

Hotfix → Main + Development

- Explain the purpose of each Git operation performed.

- **git init** – Initializes a new Git repository for the project.
- **git add .** – Adds all modified or new files to the staging area before committing.
- **git commit** – Saves the staged changes in the repository with a meaningful message.
- **git checkout -b** – Creates a new branch and switches to it for independent development.
- **git checkout** – Switches between branches such as main, development, and feature branches.
- **git merge** – Combines changes from one branch into another.
- **git status** – Displays modified, staged, untracked, or conflicting files.
- **Conflict Resolution** – Resolves conflicting changes when Git cannot automatically merge files.
- **git push** – Uploads local commits and branches to the remote repository.
- **git pull** – Downloads and integrates the latest changes from the remote repository.

- **git log** – Displays the project's commit history for tracking previous changes.
- **git branch** – Displays and manages the branches available in the repository

6. **Commit History Analysis**

- Present the commit history of the project.

The following commit history can be maintained for the Travel Expense Tracking and Budget Management Application. Since the assessment document requires the project to present its commit history, these are suitable meaningful commits to demonstrate in the repository.

- 8f42c1a Add Docker configuration and Compose setup
- 6d31b7e Add expense reports and budget alerts
- 4a25e9c Implement expense tracking module
- 2c18d6f Implement trip and budget management
- 1b73a4d Add user authentication and registration
- 0a91e2c Initial project setup

Commit Description

- **Initial project setup:** Created the basic project structure and configuration files.
- **Add user authentication and registration:** Implemented user registration and secure login functionality.
- **Implement trip and budget management:** Added trip creation and budget management features.
- **Implement expense tracking module:** Added expense recording, editing, categorization, and storage.
- **Add expense reports and budget alerts:** Added reports, summaries, and budget notification features.
- **Add Docker configuration and Compose setup:** Added Dockerfile and docker-compose.yml for containerized deployment.

- Explain how commit messages follow good version control practices and contribute to project maintenance.
  - **Clear and Specific:** Messages such as *"Implement expense tracking module"* clearly describe what was changed.
  - **Short and Meaningful:** Commit messages are concise and focus on one logical change.
  - **Consistent Format:** Similar wording is used across commits, making the history easy to read.
  - **Feature-Based Commits:** Each major feature, such as authentication, budget management, expense tracking, and reports, is committed separately.

- **Easy Maintenance:** Clear commit messages help developers quickly understand previous changes when debugging or updating the application.
- **Better Collaboration:** Team members can understand the work completed by other developers without examining every code change.
- **Easy Tracking and Rollback:** A well-organized commit history makes it easier to identify a problematic change and return to an earlier stable version.

7. **Docker Environment Design**
   - Explain why Docker is suitable for the assigned project.
     - **Consistent Environment:** Docker provides a consistent environment for developing and running the Travel Expense Tracking and Budget Management Application.
     - **Portability:** The application can run on different systems without requiring developers to manually configure the same software environment.
     - **Easy Deployment:** Docker packages the application and its dependencies into containers, making deployment simpler.
     - **Service Isolation:** Frontend, backend, and database components can run in separate containers, reducing dependency conflicts.
     - **Scalability:** Additional containers can be created when the number of users or application workload increases.
     - **Easy Testing:** Developers can create isolated containers to test application features without affecting the main development environment.
     - **Maintainability:** Containerized components can be updated, replaced, or redeployed independently.
     - **Suitable for the Assessment:** Docker is specifically included in the assignment to demonstrate containerization, deployment, portability, scalability, and maintainability.
   - Identify the application components that require containerization.

     ❖ **Frontend Application:** Containerizes the user interface used for login, trip management, expense tracking, budgets, and reports.

     ❖ **Backend Application:** Containerizes the server, REST APIs, authentication, business logic, budget calculations, and report generation.

     ❖ **Database:** Runs the application's database in a separate container for storing users, trips, budgets, and expense records.

     ❖ **External Services:** Notification or cloud services can be connected through APIs but do not necessarily need to be containerized.

- **Docker Compose:** Manages the frontend, backend, and database containers and their communication.

8. **Dockerfile Development**
   - Design and explain the Dockerfile required to containerize the application.

     For the Travel Expense Tracking and Budget Management Application, the Dockerfile can be used to containerize the backend application.

     ```
     # Use Node.js as the base image
     FROM node:20
     # Set the working directory
     WORKDIR /app
     # Copy package files
     COPY package*.json ./
     # Install dependencies
     RUN npm install
     # Copy application source code
     COPY . .
     # Expose the application port
     EXPOSE 5000
     # Start the application
         CMD ["npm", "start"]
     ```

     Purpose of Each Dockerfile Instruction

     - **FROM node:20** – Provides the Node.js environment required to run the backend.
     - **WORKDIR /app** – Creates /app as the working directory inside the container.
     - **COPY package*.json ./** – Copies the dependency configuration files into the container.
     - **RUN npm install** – Installs the required project dependencies.
     - **COPY . .** – Copies the backend source code into the container.
     - **EXPOSE 5000** – Documents the port used by the backend application.
     - **CMD ["npm", "start"]** – Starts the backend application when the container runs.

   - Describe the purpose of each instruction used in the Dockerfile.
     - **FROM node:20** – Provides the Node.js environment required to run the backend application.
     - **WORKDIR /app** – Sets /app as the working directory inside the Docker container.

- **COPY package*.json ./** – Copies the package configuration files required for installing dependencies.
- **RUN npm install** – Installs all required Node.js dependencies inside the container.
- **COPY . .** – Copies the backend application source code into the container.
- **EXPOSE 5000** – Indicates that the application uses port **5000** for communication.

9. **Docker Compose Design**
    - Develop a Docker Compose configuration for the project.

Docker Compose Configuration

For the Travel Expense Tracking and Budget Management Application**, Docker Compose can manage the** backend and database **as separate services.**

```
version: '3.8'
services:
  backend:
    build: ./backend
    container_name: travel-expense-backend
    ports:
      - "5000:5000"
    environment:
      DB_HOST: database
      DB_PORT: 5432
      DB_NAME: travel_expense
      DB_USER: admin
      DB_PASSWORD: password
    depends_on:
      - database
  database:
    image: postgres:16
    container_name: travel-expense-database
    environment:
      POSTGRES_DB: travel_expense
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: password
    ports:
      - "5432:5432"
```

```
    volumes:
      - expense_data:/var/lib/postgresql/data
volumes:
    expense_data:
```

Explanation

- **Backend Service:** Builds and runs the application's backend using the Dockerfile.
- **Database Service:** Runs PostgreSQL to store users, trips, budgets, and expense records.
- **Ports:** Allows communication with the backend through port **5000** and PostgreSQL through **5432**.
- **Environment Variables:** Provide database connection details to the backend and database containers.
- **depends_on:** Ensures the backend service starts after the database service is started.
- **Volume:** expense_data provides persistent database storage even when the database container is recreated.
- **Docker Network:** Docker Compose automatically creates a network so the backend can communicate with the database using the service name database.

- Explain the services, networking, volumes, environment variables, and dependencies used.
  - **Services:** The backend service runs the Travel Expense Tracking application, while the database service runs PostgreSQL for storing users, trips, budgets, and expenses.
  - **Networking:** Docker Compose creates a private network that allows the backend to communicate with the database using the service name database.
  - **Volumes:** The expense_data volume provides persistent storage for PostgreSQL data, preventing data loss when the database container is recreated.
  - **Environment Variables:** Variables such as DB_HOST, DB_PORT, DB_NAME, DB_USER, and DB_PASSWORD provide the backend with the required database connection details.
  - **Dependencies:** The depends_on option ensures that the backend service depends on the database service and starts after the database container is started.

10. **Container Deployment and Testing**
    - Demonstrate the execution of the application using Docker containers.

Docker Container Execution

The application can be executed using Docker Compose with the following steps:

1. **Open the Project Directory**

   cd Travel-Expense-Management

2. **Build the Docker Images**

   docker compose build

   This builds the application image using the Dockerfile.

3. **Start the Containers**

   docker compose up -d

   This starts the backend and database containers in the background.

4. **Check Running Containers**

   docker compose ps

   The backend and database services should show a **running** status.

5. **View Application Logs**

   docker compose logs

   This helps verify that the application and database have started correctly.

6. **Access the Application**

   Open a browser and access:

   http://localhost:5000

7. **Stop the Containers**

   docker compose down

   This stops and removes the running containers while the database volume remains available.

- Explain the testing procedure and provide evidence that the application runs successfully.

1. **Build the Containers**

   docker compose build

   This verifies that the Dockerfile and application dependencies are configured correctly.

2. **Start the Application**

   docker compose up -d

   This starts the backend and database containers.

3. **Check Container Status**

   docker compose ps

   Both the backend and database containers should display a **running** status.

4. **Check Application Logs**

   docker compose logs

The logs are checked for startup errors or database connection problems.

5. **Test Application Access**

   Open http://localhost:5000 in a browser and verify that the application loads successfully.

6. **Test Core Functions**

   Test login, trip creation, budget setting, expense recording, budget calculation, and report generation.

11. **Benefits of Git and Docker**

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

- ✓ **Collaboration:** Git allows multiple developers to work on different branches and merge their changes systematically. Docker provides the same application environment for all team members, reducing environment-related conflicts.
- ✓ **Version Management:** Git records every change through commits, branches, and history, making it easy to track and restore previous versions of the project.
- ✓ **Portability:** Docker packages the application with its required dependencies, allowing the Travel Expense Tracking and Budget Management Application to run consistently on different systems.
- ✓ **Deployment:** Docker simplifies deployment by providing ready-to-run containers for the application and database, reducing manual configuration.
- ✓ **Scalability:** Git supports organized development of new features, while Docker allows additional application containers to be created when the workload increases.
- ✓ **Maintainability:** Git provides a clear history of changes, while Docker keeps application components isolated and easier to update, test, and maintain.

Overall, Git manages the source code and development process, while Docker manages the application environment and deployment, making the project more organized, portable, and maintainable. These benefits directly align with the assessment requirement covering collaboration, version management, portability, deployment, scalability, and maintainability.

12. **Challenges and Improvements**

- Identify the challenges encountered during implementation.
  - **Git Configuration:** Setting up the repository, branches, and maintaining a proper commit history can be challenging during initial development.

- **Merge Conflicts:** Changes made by different developers to the same files may result in conflicts that require manual resolution.
- **Docker Configuration:** Configuring the Dockerfile, dependencies, ports, and container environment correctly can require additional troubleshooting.
- **Container Communication:** Establishing proper communication between the backend and database containers may cause configuration issues.
- **Database Connectivity:** Incorrect database credentials, environment variables, or connection settings can prevent the application from connecting to the database.
- **Environment Differences:** Differences between local development environments and Docker containers may cause application or dependency-related errors.
- **Testing and Debugging:** Identifying and resolving errors after containerization requires additional testing and log analysis.

- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.
  - **Git Branching:** Follow a consistent branching strategy with main, development, feature, release, and hotfix branches.
  - **Commit Practices:** Use clear, meaningful commit messages and make small, focused commits for easier tracking and maintenance.
  - **Code Review:** Introduce pull requests and peer reviews before merging changes into the main branch.
  - **Automated Testing:** Add automated tests to verify application functionality before merging or deploying changes.
  - **CI/CD Integration:** Use continuous integration and deployment pipelines to automatically build, test, and deploy the application.
  - **Docker Optimization:** Use smaller base images and multi-stage builds to reduce Docker image size and improve deployment efficiency.
  - **Security Improvements:** Store sensitive configuration in secure environment variables or secret-management systems instead of exposing credentials.
  - **Monitoring and Logging:** Add container health checks, centralized logging, and monitoring to identify failures quickly.
  - **Future Enhancements:** The deployment can later support container orchestration and automated scaling as the application grows.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25